Tugas Pertemuan 2	
IF21W0508 – Praktikum Web	
NPM	243040055
Nama	Alisha Nur Salampessy
Kelas	F
Link Github Repository	PW2026_243040055/pertemuan2 at main · Alishanursalampessy/PW2026_243040055

Bagian 1. Screenshot Semua Kode

Tugas Pertemuan 2 terdiri atas tiga tugas: **Tugas 1** (filter daftar tugas), **Tugas 2** (hapus semua tugas selesai), dan **Tugas 3** (aplikasi Catatan Keuangan Mini). Kodenya berada di folder **src/**. Pada **DaftarTugas.tsx**, baris yang disorot kuning adalah kode Tugas 1 dan Tugas 2. Berkas yang panjang dipotong menjadi beberapa gambar berurutan.

src/App.tsx – Kerangka: tab pemilih halaman (menghubungkan Tugas 1-2 dengan Tugas 3)

```tsx
// src/App.tsx (kerangka: memilih halaman yang tampil)
import { useState } from "react";
import "./App.css";
import DaftarTugas from "./DaftarTugas";
import CatatanKeuangan from "./keuangan/CatatanKeuangan";

type Halaman = "tugas" | "keuangan";

function App() {
  const [halaman, setHalaman] = useState<Halaman>("tugas");

  return (
    <div className="wadah">
      <div className="tab">
        <button
          className={`tombol-filter ${halaman === "tugas" ? "aktif" : ""}`}
          onClick={() => setHalaman("tugas")}
        >
          Daftar Tugas
        </button>
        <button
          className={`tombol-filter ${halaman === "keuangan" ? "aktif" : ""}`}
          onClick={() => setHalaman("keuangan")}
        >
          Catatan Keuangan
        </button>
      </div>

      {halaman === "tugas" ? <DaftarTugas /> : <CatatanKeuangan />}
    </div>
  );
}

export default App;

```

src/DaftarTugas.tsx – Tugas 1: filter dan Tugas 2: hapus semua yang selesai

```tsx
// src/DaftarTugas.tsx (komponen induk aplikasi Daftar Tugas Praktikum)
import { useState, useEffect } from "react";
import Kartu from "./Kartu";
import Penghitung from "./Penghitung";
import FormTugas from "./FormTugas";
import ItemTugas from "./ItemTugas";
import type { Tugas } from "./ItemTugas";

type Filter = "semua" | "belum" | "selesai";

const tugasAwal: Tugas[] = [
  { id: 1, judul: "Install Node.js dan Vite", selesai: true },
  { id: 2, judul: "Pelajari useState", selesai: false },
];

function DaftarTugas() {
  // 1. State utama: coba ambil dari localStorage, jika kosong pakai data contoh
  const [daftarTugas, setDaftarTugas] = useState<Tugas[]>(() => {
    try {
      const dataTersimpan = localStorage.getItem("daftar-tugas");
      if (dataTersimpan) {
        return JSON.parse(dataTersimpan) as Tugas[]; // ubah teks JSON kembali menjadi array
      }
    } catch {
      // isi localStorage rusak: abaikan dan pakai data contoh
    }
    return tugasAwal;
  });

  // Tugas 1: state untuk pilihan filter
  const [filter, setFilter] = useState<Filter>("semua");

  // Efek A: dijalankan SEKALI saat komponen pertama kali tampil
  useEffect(() => {
    console.log("Aplikasi Daftar Tugas siap digunakan!");
  }, []);

  // Efek B: perbarui judul tab peramban setiap daftarTugas berubah
  useEffect(() => {
    const sisa = daftarTugas.filter((tugas) => !tugas.selesai).length;
    document.title = `(${sisa}) Tugas Belum Selesai`;
  }, [daftarTugas]);

  // Efek C: simpan daftarTugas ke localStorage setiap kali berubah
  useEffect(() => {
    localStorage.setItem("daftar-tugas", JSON.stringify(daftarTugas));
  }, [daftarTugas]);

  // 2. Menambah tugas: buat array BARU berisi tugas lama + tugas baru
  const tambahTugas = (judul: string) => {
    const tugasBaru: Tugas = { id: Date.now(), judul: judul, selesai: false };
    setDaftarTugas([...daftarTugas, tugasBaru]);
  };

  // 3. Mengubah status selesai: salin tugas yang dicentang, biarkan yang lain
  const ubahStatus = (id: number) => {
    const daftarBaru = daftarTugas.map((tugas) => {
      if (tugas.id === id) {
        return { ...tugas, selesai: !tugas.selesai }; // salinan dengan status dibalik
      }
      return tugas; // tugas lain tidak berubah
    });
    setDaftarTugas(daftarBaru);
  };

  // 4. Menghapus tugas: ambil semua tugas KECUALI yang id-nya sama
  const hapusTugas = (id: number) => {
    setDaftarTugas(daftarTugas.filter((tugas) => tugas.id !== id));
  };

  // Tugas 2: hapus semua tugas yang berstatus selesai
  const hapusSemuaSelesai = () => {
    setDaftarTugas(daftarTugas.filter((tugas) => !tugas.selesai));
  };

  // 5. Nilai turunan: dihitung dari state, tidak perlu state baru
  const jumlahSelesai = daftarTugas.filter((tugas) => tugas.selesai).length;
```

```
78
79    // Tugas 1: saring daftar sesuai filter SEBELUM di-map
80    const tugasTampil = daftarTugas.filter((tugas) => {
81      if (filter === "belum") return !tugas.selesai;
82      if (filter === "selesai") return tugas.selesai;
83      return true; // "semua"
84    });
85
86    const pilihanFilter: { nilai: Filter; label: string }[] = [
87      { nilai: "semua", label: "Semua" },
88      { nilai: "belum", label: "Belum Selesai" },
89      { nilai: "selesai", label: "Selesai" },
90    ];
91
92    return (
93      <>
```

```jsx
      <h1 className="judul">Daftar Tugas Praktikum</h1>
      <p className="subjudul">Laboratorium Informatika - Modul 2: React Lanjutan</p>

      <Penghitung />

      <Kartu judul="Tambah Tugas Baru">
        <FormTugas onTambah={tambahTugas} />
      </Kartu>

      <Kartu judul="Daftar Tugas">
        <p className="ringkasan">
          {jumlahSelesai} dari {daftarTugas.length} tugas selesai
        </p>

        {/* Tugas 1: tiga tombol filter, yang aktif tampil berbeda */}
        <div className="filter">
          {pilihanFilter.map((p) => (
            <button
              key={p.nilai}
              className={`tombol-filter ${filter === p.nilai ? "aktif" : ""}`}
              onClick={() => setFilter(p.nilai)}
            >
              {p.label}
            </button>
          ))}
        </div>

        {/* Conditional rendering dengan ternary: tampilkan pesan jika kosong */}
        {daftarTugas.length === 0 ? (
          <p className="kosong">Belum ada tugas. Yuk tambahkan tugas pertamamu!</p>
        ) : tugasTampil.length === 0 ? (
          <p className="kosong">Tidak ada tugas pada filter ini.</p>
        ) : (
          <ul className="daftar">
            {tugasTampil.map((tugas) => (
              <ItemTugas
                key={tugas.id}
                tugas={tugas}
                onUbahStatus={ubahStatus}
                onHapus={hapusTugas}
              />
            ))}
          </ul>
        )}
```

```
            )}

            {/* Tugas 2: tombol hanya muncul jika ada minimal satu tugas selesai */}
            {jumlahSelesai > 0 && (
              <div className="aksi">
                <button className="tombol tombol-merah" onClick={hapusSemuaSelesai}>
                  Hapus Semua yang Selesai
                </button>
              </div>
            )}
          </Kartu>
        </>
      );
    }

    export default DaftarTugas;
```

src/keuangan/CatatanKeuangan.tsx – Tugas 3: komponen induk Catatan Keuangan Mini

```tsx
// src/keuangan/CatatanKeuangan.tsx (komponen induk Catatan Keuangan Mini)
import { useState, useEffect } from "react";
import Kartu from "../Kartu";
import FormTransaksi from "./FormTransaksi";
import ItemTransaksi from "./ItemTransaksi";
import { rupiah } from "./format";
import type { Transaksi } from "./ItemTransaksi";

function CatatanKeuangan() {
  // State utama: ambil dari localStorage jika ada
  const [daftar, setDaftar] = useState<Transaksi[]>(() => {
    try {
      const data = localStorage.getItem("catatan-keuangan");
      if (data) return JSON.parse(data) as Transaksi[];
    } catch {
      // isi localStorage rusak: mulai dari kosong
    }
    return [];
  });

  // Efek: simpan ke localStorage setiap daftar berubah
  useEffect(() => {
    localStorage.setItem("catatan-keuangan", JSON.stringify(daftar));
  }, [daftar]);

  const tambah = (data: Omit<Transaksi, "id">) => {
    setDaftar([...daftar, { id: Date.now(), ...data }]); // array baru
  };

  const hapus = (id: number) => {
    setDaftar(daftar.filter((t) => t.id !== id)); // array baru tanpa item itu
  };

  // Nilai turunan (dihitung dari state, bukan disimpan sebagai state baru)
  const totalMasuk = daftar
    .filter((t) => t.jenis === "pemasukan")
    .reduce((jumlah, t) => jumlah + t.nominal, 0);
  const totalKeluar = daftar
    .filter((t) => t.jenis === "pengeluaran")
    .reduce((jumlah, t) => jumlah + t.nominal, 0);
  const saldo = totalMasuk - totalKeluar;

  return (
    <>
```

```jsx
      <h1 className="judul">Catatan Keuangan Mini</h1>
      <p className="subjudul">Laboratorium Informatika - Modul 2: Tugas Terstruktur</p>

      <Kartu judul="Saldo Saat Ini">
        <p className={`saldo ${saldo < 0 ? "pengeluaran" : "pemasukan"}`}>{rupiah(saldo)}</p>
        <div className="rekap">
          <span className="pemasukan">Masuk: {rupiah(totalMasuk)}</span>
          <span className="pengeluaran">Keluar: {rupiah(totalKeluar)}</span>
        </div>
      </Kartu>

      <Kartu judul="Tambah Transaksi">
        <FormTransaksi onTambah={tambah} />
      </Kartu>

      <Kartu judul="Daftar Transaksi">
        {daftar.length === 0 ? (
          <p className="kosong">Belum ada transaksi. Yuk catat transaksi pertamamu!</p>
        ) : (
          <ul className="daftar">
            {daftar.map((t) => (
              <ItemTransaksi key={t.id} transaksi={t} onHapus={hapus} />
            ))}
          </ul>
        )}
      </Kartu>
    </>
  );
}

export default CatatanKeuangan;

```

src/keuangan/FormTransaksi.tsx – Tugas 3: formulir transaksi

```tsx
// src/keuangan/FormTransaksi.tsx
import { useState } from "react";
import type { FormEvent } from "react";
import type { Transaksi } from "./ItemTransaksi";

type DataBaru = Omit<Transaksi, "id">; // transaksi tanpa id (id dibuat di induk)

type FormTransaksiProps = {
  onTambah: (data: DataBaru) => void;
};

function FormTransaksi({ onTambah }: FormTransaksiProps) {
  // Tiga input terkendali: nilainya selalu sama dengan state
  const [keterangan, setKeterangan] = useState("");
  const [nominal, setNominal] = useState(""); // input selalu berupa teks
  const [jenis, setJenis] = useState<Transaksi["jenis"]>("pemasukan");

  // Number(...) mengubah teks menjadi angka sebelum dipakai
  const angka = Number(nominal);
  const tidakValid = keterangan.trim() === "" || nominal === "" || !(angka > 0);

  const tanganiKirim = (event: FormEvent<HTMLFormElement>) => {
    event.preventDefault();
    if (tidakValid) return;

    onTambah({ keterangan: keterangan.trim(), nominal: angka, jenis });
    setKeterangan("");
    setNominal("");
  };

  return (
    <form onSubmit={tanganiKirim} className="form-kolom">
      <input
        className="input"
        type="text"
        placeholder="Keterangan (contoh: Uang saku)"
        value={keterangan}
        onChange={(e) => setKeterangan(e.target.value)}
      />
      <div className="baris">
        <input
          className="input"
          type="number"
          placeholder="Nominal"
          value={nominal}
          onChange={(e) => setNominal(e.target.value)}
        />
        <select
          className="pilih"
          value={jenis}
          onChange={(e) => setJenis(e.target.value as Transaksi["jenis"])}
        >
          <option value="pemasukan">Pemasukan</option>
          <option value="pengeluaran">Pengeluaran</option>
        </select>
      </div>
      <button className="tombol" type="submit" disabled={tidakValid}>
        Tambah Transaksi
      </button>
      {nominal !== "" && !(angka > 0) && (
        <p className="galat">Nominal harus lebih besar dari 0.</p>
      )}
    </form>
  );
}

export default FormTransaksi;

```

src/keuangan/ItemTransaksi.tsx – Tugas 3: satu baris transaksi

```tsx
// src/keuangan/ItemTransaksi.tsx
import { rupiah } from "./format";

export type Transaksi = {
  id: number;
  keterangan: string;
  nominal: number; // sudah berupa angka
  jenis: "pemasukan" | "pengeluaran";
};

type ItemTransaksiProps = {
  transaksi: Transaksi;
  onHapus: (id: number) => void;
};

function ItemTransaksi({ transaksi, onHapus }: ItemTransaksiProps) {
  const pemasukan = transaksi.jenis === "pemasukan";

  return (
    <li className="item">
      <span>{transaksi.keterangan}</span>

      {/* Pemasukan berwarna hijau, pengeluaran berwarna merah */}
      <span className={pemasukan ? "pemasukan" : "pengeluaran"}>
        {pemasukan ? "+" : "-"} {rupiah(transaksi.nominal)}
      </span>

      <button
        className="tombol tombol-merah tombol-kecil"
        onClick={() => onHapus(transaksi.id)}
      >
        Hapus
      </button>
    </li>
  );
}

export default ItemTransaksi;

```

src/keuangan/format.ts – Tugas 3: fungsi format rupiah

```ts
// src/keuangan/format.ts
// Mengubah angka menjadi tulisan rupiah, contoh: 50000 -> "Rp 50.000"
export const rupiah = (angka: number) => "Rp " + angka.toLocaleString("id-ID");

```

src/App.css – Gaya tambahan untuk Tugas 1-3

```css
/* src/App.css */
.wadah { max-width: 640px; margin: 40px auto; padding: 0 20px; font-family: Arial, sans-serif; color: #1F2937; }
.judul { color: #1B365D; margin-bottom: 4px; }
.subjudul { color: #6B7280; margin-top: 0; margin-bottom: 24px; }
.kartu { background: #FFFFFF; border: 1px solid #D0D7DE; border-radius: 10px; padding: 20px; margin-bottom: 20px; box-shadow: 0
.kartu h2 { margin-top: 0; font-size: 18px; color: #1B365D; }
.baris { display: flex; gap: 8px; }
.input { flex: 1; padding: 10px; border: 1px solid #D0D7DE; border-radius: 6px; font-size: 14px; }
.tombol { padding: 10px 16px; border: none; border-radius: 6px; background: #1B365D; color: #FFFFFF; font-weight: bold; cursor:
.tombol:disabled { background: #9CA3AF; cursor: not-allowed; }
.tombol-merah { background: #DC2626; }
.tombol-kecil { padding: 6px 10px; font-size: 12px; }
.daftar { list-style: none; padding: 0; margin: 0; }
.item { display: flex; align-items: center; gap: 10px; padding: 10px 0; border-bottom: 1px solid #E5E7EB; }
.item span { flex: 1; }
.selesai { text-decoration: line-through; color: #9CA3AF; }
.kosong { color: #6B7280; text-align: center; font-style: italic; }
.galat { color: #DC2626; font-size: 13px; margin: 6px 0 0; }
.ringkasan { font-size: 14px; color: #374151; }

/* Tambahan untuk Tugas 1-3 */
.tab { display: flex; gap: 8px; margin-bottom: 20px; }
.tombol-filter { padding: 8px 14px; border: 1px solid #D0D7DE; border-radius: 6px; background: #FFFFFF; color: #374151; cursor
.tombol-filter.aktif { background: #1B365D; border-color: #1B365D; color: #FFFFFF; font-weight: bold; }
.filter { display: flex; gap: 8px; margin-bottom: 12px; }
.aksi { margin-top: 14px; }
.pemasukan { color: #15803D; font-weight: bold; }
.pengeluaran { color: #DC2626; font-weight: bold; }
.saldo { font-size: 28px; font-weight: bold; margin: 4px 0 0; }
.rekap { display: flex; gap: 16px; font-size: 14px; margin-top: 6px; }
.pilih { padding: 10px; border: 1px solid #D0D7DE; border-radius: 6px; font-size: 14px; background: #FFFFFF; }
.form-kolom { display: grid; gap: 8px; }

```

Bagian 2. Penjelasan Kode

Aplikasi dibuat dengan React dan TypeScript. Tiap komponen menerima data lewat *props* yang tipenya dideklarasikan, menyimpan data yang berubah dengan *state* (`useState`), dan menyimpan data ke `localStorage` lewat `useEffect`. Komponen `Kartu` dari latihan modul dipakai ulang sebagai pembungkus tampilan, tetapi kodenya tidak dibahas di laporan ini.

App.tsx

- `type Halaman = "tugas" | "keuangan"` membatasi nilai state hanya pada dua halaman. State `halaman` dibuat dengan `useState<Halaman>("tugas")` sehingga halaman awal adalah Daftar Tugas.
- Dua tombol tab memanggil `setHalaman` saat diklik. Tab yang aktif diberi kelas `aktif` lewat template literal pada `className`.
- Conditional rendering dengan ternary: `halaman === "tugas" ? <DaftarTugas /> : <CatatanKeuangan />`. Hanya satu halaman yang tampil pada satu waktu.

DaftarTugas.tsx – Tugas 1: Filter

- `type Filter = "semua" | "belum" | "selesai"` dan state `filter` (nilai awal `"semua"`) menyimpan pilihan filter yang sedang aktif.
- Variabel `tugasTampil` dihitung dengan `daftarTugas.filter(...)` sebelum di-`map`. Array asli `daftarTugas` tidak diubah, sehingga ringkasan "x dari y tugas selesai", penghapusan, dan penyimpanan ke `localStorage` tetap memakai data lengkap.
- Array `pilihanFilter` berisi nilai dan label tiap tombol. Tiga tombol dirender dengan `.map()` dan `key={p.nilai}`. Tombol aktif mendapat kelas `aktif`, dan `onClick` memanggil `setFilter(p.nilai)`.
- Ternary bersarang menampilkan pesan yang berbeda: "Belum ada tugas…" jika `daftarTugas` kosong, dan "Tidak ada tugas pada filter ini." jika daftar ada tetapi hasil filter kosong.

DaftarTugas.tsx – Tugas 2: Hapus Semua yang Selesai

- Fungsi `hapusSemuaSelesai` memanggil `setDaftarTugas(daftarTugas.filter((tugas) => !tugas.selesai))`. Hasilnya array baru yang hanya berisi tugas belum selesai; array lama tidak dimodifikasi.
- Tombol dirender dengan `jumlahSelesai > 0 && (...)`, sehingga hanya muncul jika ada minimal satu tugas selesai. `jumlahSelesai` adalah nilai turunan yang dihitung dari state.
- Karena efek penyimpanan bergantung pada `[daftarTugas]`, hasil penghapusan otomatis tersimpan ke `localStorage` dan judul tab browser ikut diperbarui oleh efek yang sudah ada.

keuangan/format.ts

- Fungsi `rupiah(angka)` mengubah angka menjadi teks berformat Indonesia dengan `toLocaleString("id-ID")`, misalnya 50000 menjadi "Rp 50.000". Fungsi ini dipakai di beberapa komponen agar format seragam.

keuangan/ItemTransaksi.tsx – Tugas 3

- Tipe `Transaksi` didefinisikan dan diekspor di sini: `id`, `keterangan`, `nominal` (angka), dan `jenis` yang dibatasi `"pemasukan" | "pengeluaran"`.
- Props: `transaksi` dan `onHapus`. Kelas `pemasukan` (hijau) atau `pengeluaran` (merah) dan tanda "+" atau "-" dipilih berdasarkan `jenis`.
- Tombol Hapus memanggil `onHapus(transaksi.id)` lewat arrow function agar fungsi tidak langsung terpanggil saat render.

keuangan/FormTransaksi.tsx – Tugas 3

- Tiga *controlled input* (`keterangan`, `nominal`, `jenis`) nilainya selalu sama dengan state. `nominal` disimpan sebagai teks karena input selalu bernilai teks, lalu diubah dengan `Number(nominal)`.
- Validasi: `tidakValid` bernilai benar jika keterangan kosong, nominal kosong, atau nominal tidak lebih dari 0. Tombol memakai `disabled={tidakValid}`, dan pesan “Nominal harus lebih besar dari 0.” tampil jika nominal terisi tetapi tidak valid.
- `tanganiKirim` memanggil `event.preventDefault()` agar halaman tidak reload, lalu menjalankan `onTambah` (fungsi dari induk) dan mengosongkan input. Tipe `Omit<Transaksi, "id">` dipakai karena `id` dibuat oleh komponen induk.

keuangan/CatatanKeuangan.tsx – Tugas 3

- State `daftar` diisi dari `localStorage` lewat fungsi inisialisasi di `useState` (dibungkus `try/catch` agar isi yang rusak tidak membuat aplikasi gagal). `useEffect` dengan dependensi `[daftar]` menyimpan ulang data setiap daftar berubah.
- `tambah` membuat array baru dengan `[...daftar, { id: Date.now(), ...data }]`, sedangkan `hapus` memakai `filter` untuk membuat array tanpa item tersebut. State tidak pernah diubah langsung.
- `totalMasuk` dan `totalKeluar` dihitung dari `daftar` dengan `filter` dan `reduce`; `saldo` adalah selisihnya. Ketiganya nilai turunan, bukan state baru. Saldo berwarna merah jika kurang dari 0 dan hijau jika tidak.
- Jika `daftar` kosong tampil pesan khusus; jika tidak, daftar dirender dengan `.map()` dan `key={t.id}`.

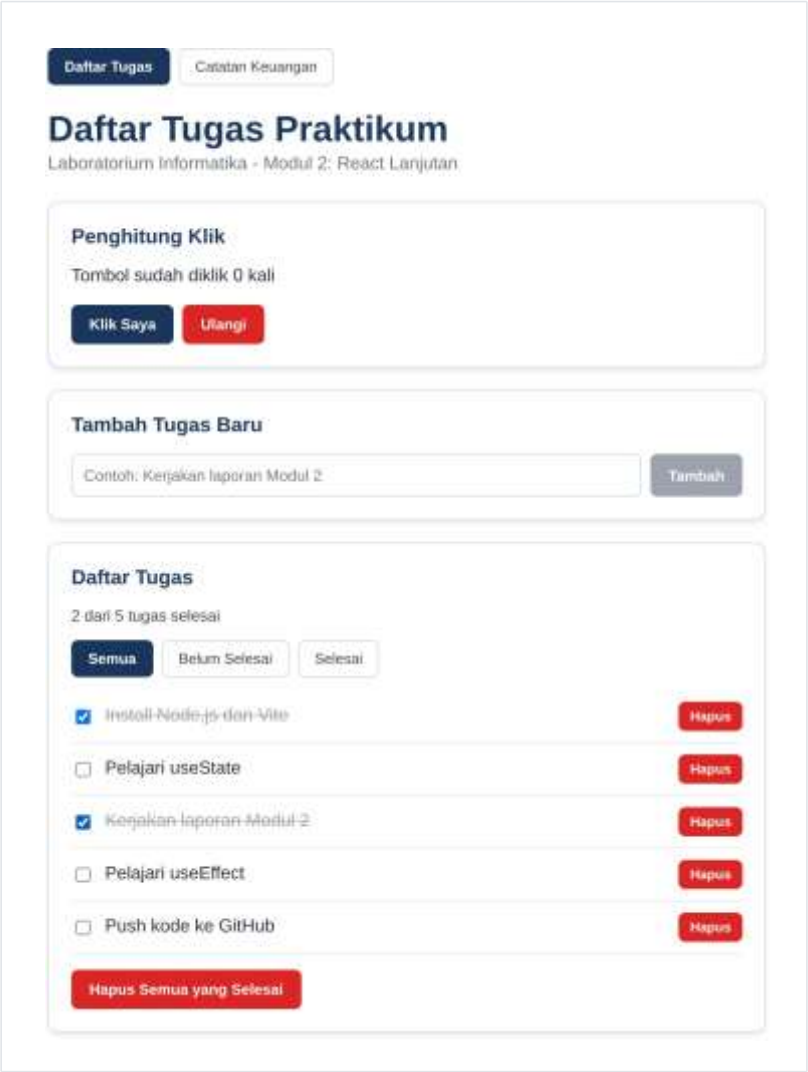
App.css (bagian tambahan Tugas 1-3)

- Menambah kelas untuk tab (`.tab`, `.tombol-filter`, `.aktif`), filter, tombol aksi, warna pemasukan/pengeluaran, tampilan saldo (`.saldo`, `.rekap`), dan tata letak formulir (`.pilih`, `.form-kolom`).

Bagian 3. Screenshot Hasil Antarmuka

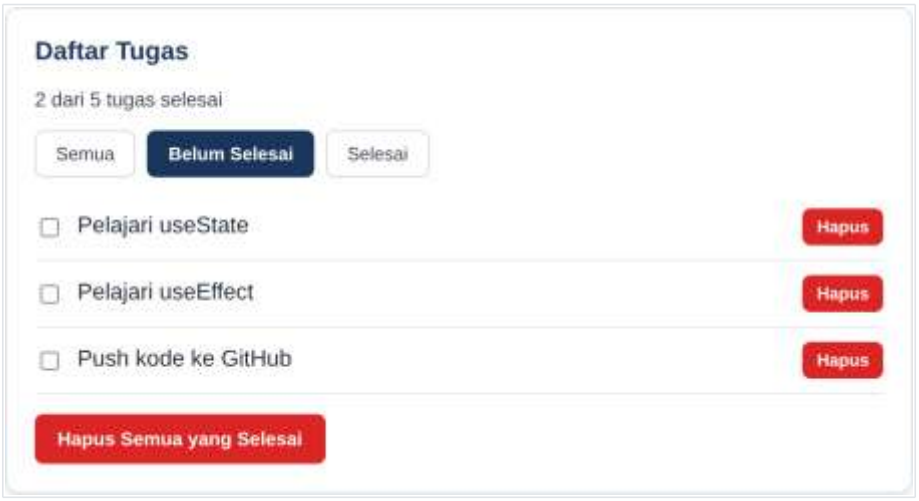
Tangkapan layar diambil dari aplikasi yang dijalankan di browser, dengan data uji yang dimasukkan lewat antarmuka.

Halaman Daftar Tugas dan tab pemilih halaman

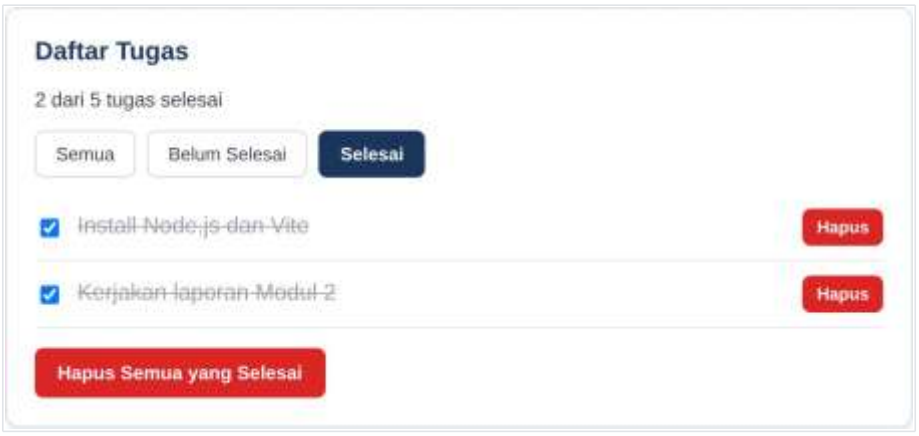


Gambar 1. Halaman Daftar Tugas (2 dari 5 tugas selesai) dengan tombol “Hapus Semua yang Selesai”

Tugas 1 – Filter daftar tugas

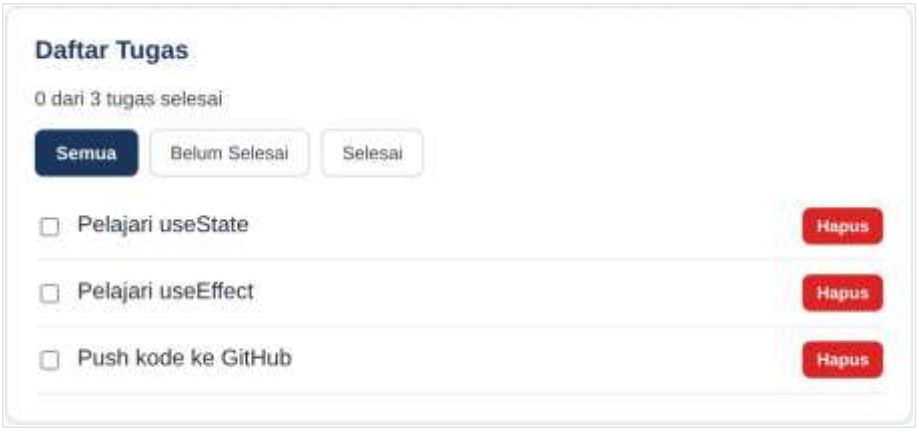


Gambar 2. Filter “Belum Selesai” aktif: hanya 3 tugas yang tampil



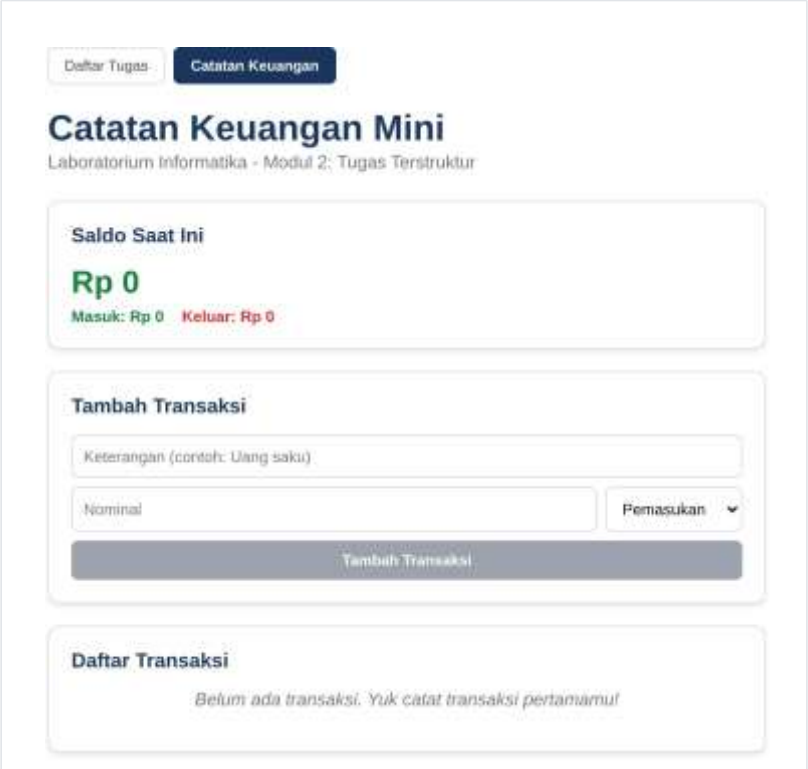
Gambar 3. Filter “Selesai” aktif: hanya 2 tugas yang tampil

Tugas 2 – Hapus semua tugas yang selesai



Gambar 4. Setelah tombol diklik: 2 tugas selesai terhapus, 3 tugas tersisa, dan tombol hilang

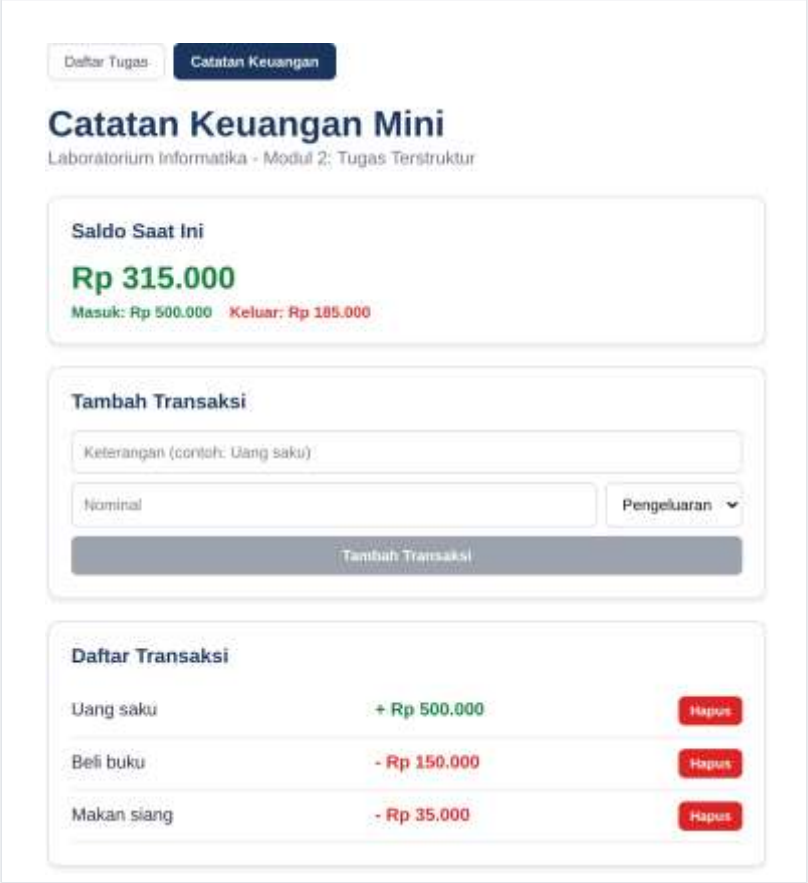
Tugas 3 – Catatan Keuangan Mini



Gambar 5. Kondisi awal: saldo Rp 0, daftar kosong, dan tombol Tambah Transaksi nonaktif



Gambar 6. Validasi: nominal 0 menampilkan pesan galat dan tombol tetap nonaktif



Gambar 7. Setelah tiga transaksi: saldo Rp 315.000 (masuk Rp 500.000, keluar Rp 185.000)

Bagian 4. Penjelasan Hasil Antarmuka

Hasil pengujian di browser, dari atas ke bawah:

- **Tab halaman**: tombol “Daftar Tugas” dan “Catatan Keuangan” berpindah halaman; tab yang aktif berwarna biru tua.
- **Tugas 1 (filter)**: dengan 5 tugas (2 selesai dan 3 belum), filter “Semua” menampilkan 5 tugas, “Belum Selesai” menampilkan 3, dan “Selesai” menampilkan 2. Ringkasan “2 dari 5 tugas selesai” tidak berubah saat filter diganti karena dihitung dari data lengkap.
- **Tugas 2 (hapus semua selesai)**: tombol merah “Hapus Semua yang Selesai” muncul karena ada tugas selesai. Setelah diklik, 2 tugas selesai terhapus, 3 tugas tersisa, tombol hilang, dan judul tab browser menjadi “(3) Tugas Belum Selesai”. Setelah halaman di-refresh, 3 tugas itu tetap ada karena tersimpan di `localStorage`.
- **Tugas 3 (catatan keuangan)**: awalnya saldo Rp 0 dan muncul pesan “Belum ada transaksi”. Tombol Tambah Transaksi nonaktif sampai keterangan dan nominal valid; nominal 0 memunculkan pesan galat.
- Setelah transaksi Uang saku (pemasukan Rp 500.000), Beli buku (pengeluaran Rp 150.000), dan Makan siang (pengeluaran Rp 35.000), saldo menjadi Rp 315.000. Pemasukan berwarna hijau dengan tanda “+”, pengeluaran berwarna merah dengan tanda “-”.
- Menghapus transaksi Beli buku membuat saldo menjadi Rp 465.000, dan 2 transaksi yang tersisa tetap ada setelah refresh. Ketika pengeluaran lebih besar dari pemasukan, saldo bernilai negatif dan tampil merah.
- Selama pengujian, console browser tidak menampilkan pesan error.